

535235 Physical Design Automation

Lab3: Optimizer and Legalizer Co-optimization

313552038 饒景鈞

1. How to compile and execute my program.

- Compile:
 - \$ tar xvf 313552038.tar
 - \$ cd 313552038
 - \$ make -j
- Excute:
 - \$./Legalizer {*.lg} {*.opt} {*_post.lg}

2. The total distance and run time of each testcase.

	testcase1_16900	testcase1_ALL0_5000	testcase2_100
Move Times	0	0	0
Total Distance	683367840.00	49475212800.00	12566920200.00
Total CPU Time	41.22s	294.51s	8.72s

```
[2024PDA053@mseda01 Lab3]$ time ./Legalizer input/testcase1_16900.lg input/testcase1_16900.opt output/testcase1_16900.lg
40.473u 0.749s 1:31.76 44.9% 0+0k 8112+14304io 0pf+0w
[2024PDA053@mseda01 Lab3]$ ./Evaluator input/testcase1_16900.lg input/testcase1_16900.opt output/testcase1_16900.lg
```

Cost	-	Weight	Value	Percentage(%)
Move Times	0.00	10000.00	0.00	0.00(%)
Total Distance	683367840.00	1.00	683367840.00	100.00(%)
Total	-	-	683367840.00	100.00(%)

```
[2024PDA053@mseda01 Lab3]$ time ./Legalizer input/testcase1_ALL0_5000.lg input/testcase1_ALL0_5000.opt output/testcase1_ALL0_5000.lg
293.337u 1.169s 7:27.83 65.7% 0+0k 8400+27456io 0pf+0w
[2024PDA053@mseda01 Lab3]$ ./Evaluator input/testcase1_ALL0_5000.lg input/testcase1_ALL0_5000.opt output/testcase1_ALL0_5000.lg
```

Cost	-	Weight	Value	Percentage(%)
Move Times	0.00	50000.00	0.00	0.00(%)
Total Distance	2473760640.00	20.00	49475212800.00	100.00(%)
Total	-	-	49475212800.00	100.00(%)

```
[2024PDA053@mseda01 Lab3]$ time ./Legalizer input/testcase2_100.lg input/testcase2_100.opt output/testcase2_100.lg
8.464u 0.254s 0:14.48 60.1% 0+0k 0+23672io 0pf+0w
[2024PDA053@mseda01 Lab3]$ ./Evaluator input/testcase2_100.lg input/testcase2_100.opt output/testcase2_100.lg
```

Cost	-	Weight	Value	Percentage(%)
Move Times	0.00	50000.00	0.00	0.00(%)
Total Distance	628346010.00	20.00	12566920200.00	100.00(%)
Total	-	-	12566920200.00	100.00(%)

3. Implementation (with pseudo code):

My program uses a brute-force method to solve this problem:

I. **Use the input cell as the global placement**

II. **Merged of banking cell**

Do not move any cell in the global placement. Remove the cells to be merged from the placement and generate a new merged cell.

III. **Find an appropriate row and column for the merged cell**

Sort the rows based on their proximity to the desired position of the merged cell, starting from the closest row and moving to the farthest row. Begin searching from the closest row, scanning from left to right (row.startX to row.endX) to check if there is enough space to place the merged cell. If space is available, place the cell. If no suitable space is found in the current row, move to the next closest row and repeat the search. Continue this process until a valid position is found. If placing the cell exceeds the height of the die, skip the search to next row.

IV. **Insert the cell to R tree:**

Once a valid position for the merged cell has been determined, represent the merged cell as a bounding box defined by its coordinates (startX, startY, endX, endY). Insert this bounding box into the R-tree structure.

```

1  MAIN FUNCTION:
2  1. Parse input file to initialize die, cells, and rows.
3  2. Initialize R-tree with the current placement.
4  3. For each merged cell in the input:
5      a. Identify and remove the cells to be merged from R-tree and the global placement.
6      b. Create a new merged cell with updated properties.
7      c. Attempt to place the merged cell:
8          i. Legalize placement using closest row proximity logic.
9          ii. Handle error if placement is unsuccessful.
10     d. Output the placement result.
11
12  FUNCTIONS:
13  - Parse Input:
14      Read file to extract die dimensions, cell properties, and row data.
15
16  - Initialize R-tree:
17      Clear existing data in R-tree.
18      Insert all input cells as bounding boxes.
19
20  - Remove Cell from R-tree:
21      Remove a specific cell's bounding box from the R-tree.
22
23  - Insert Cell to R-tree:
24      Add a cell as a bounding box to the R-tree.
25
26  - Can Place Cell:
27      Check if a cell can be placed in a given area using R-tree overlap queries.
28
29  - Legalize Placement:
30      Sort rows by proximity to the target y-coordinate.
31      For each row:
32          a. If height exceeds die boundaries, skip to the next row.
33          b. Compute available space within the row.
34          c. Check for cell placement using left-to-right scan:
35              - If space is found, place the cell and exit.
36              - If space overlaps, adjust placement based on the bounding boxes of overlapping cells.
37      Handle cases where placement exceeds die height.
38
39  - Output:
40      Write the placement coordinates and other relevant data to the output file.

```

4. Time Complexity Analysis:

For k times banking, n cells, r rows, and row width w :

- I. **I/O input: $O(n)$**
- II. **Merged of banking cell:**
 - i. Remove the cells to be merged from the placement: $O(n)$
 - ii. Generate a new merged cell $O(1)$
- III. **Find an appropriate row and column for the merged cell**
 - i. Locate the row closest to t merged cell: $O(r)$
 - ii. Starting from that row, search from left to right: $O(w)$
 - iii. Check overlap and insert to the tree: $O(\log n)$
 - iv. Total for all rows: $O(r*w)$
- IV. **Total time of k banking: $k*O(r*w)$**

5. Discussion (Special Features & Conclusion):

The brute-force method employed in my program offers a straightforward and simple implementation for solving the placement problem. One of its key advantages is that it does not require moving any existing cells in the global placement, preserving their original positions. This design simplifies the algorithm and ensures that the placement is stable and predictable.

However, this approach comes with a few trade-offs. The cost of execution is relatively higher compared to more advanced algorithms, and the runtime performance is slower due to the exhaustive search required to find suitable positions for merged cells. Specifically, for k iterations of banking, the total time complexity is $O(k * r * w)$, where r is the number of rows and w is the row width.

In summary, while the brute-force method excels in simplicity and avoids cell movement, it incurs a higher computational cost and slower execution time, making it more suitable for scenarios where accuracy and stability are prioritized over efficiency.

6. Feedback:

Thank you, TA, for all your hard work and dedication.

